# Integridad de datos y DML

## Restricciones, claves foráneas, variables e `INSERT`, `UPDATE`, `DELETE` y `SELECT`

**Semana 07 - Sesión 13**

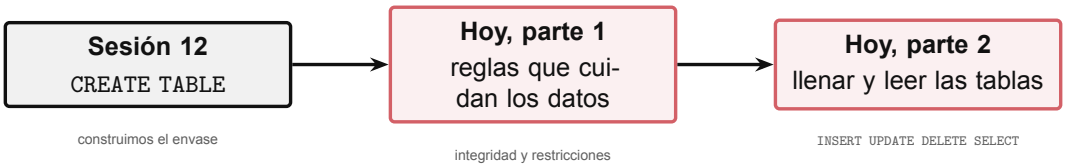
Rel Guzman



**Logro de la sesión**

Al finalizar, podrás reconocer los tres tipos de **integridad**, declarar **restricciones** y **claves foráneas** en MySQL, SQL Server y SQLite, usar **variables** y escribir tus primeros `INSERT`, `UPDATE`, `DELETE` y `SELECT`.

# Dónde estamos




**En palabras simples**

La sesión pasada construimos los estantes. Hoy ponemos los **letreros de reglas** («aquí solo libros», «uno por casillero») y empezamos a **guardar cosas** en ellos.

# Restricciones de integridad

# ¿Qué es la integridad de datos?

**Integridad**

Que los datos guardados sean **correctos y coherentes**: que no haya filas repetidas, valores imposibles ni referencias a cosas que no existen.

**Fila repetida**

Dos alumnos con el mismo código. ¿A cuál le pongo la nota?

**Valor imposible**

Un alumno con edad `-4`, o una nota de `25` sobre 20.

**Referencia rota**

Un pedido de un cliente que **no existe**.

La buena noticia: no hay que vigilarlo a mano. Se escriben **restricciones** y el gestor **rechaza** cualquier dato que las rompa.

# Tres tipos de integridad, cinco restricciones

| Tipo de integridad | Restricción | Para qué sirve |
|---|---|---|
| **Entidad** | `PRIMARY KEY` | Identificar cada fila: clave primaria |
| | `UNIQUE` | Que un valor no se repita: clave única |
| **Referencial** | `FOREIGN KEY` | Apuntar a una fila de otra tabla: clave foránea |
| **Dominio** | `DEFAULT` | Poner un valor por defecto si no se indica |
| | `CHECK` | Validar un rango, formato o lista de valores |

**Entidad:** cada fila es **una sola cosa**, distinguible de las demás.

**Referencial:** las conexiones entre tablas **no se rompen**.

**Dominio:** cada valor está **dentro de lo permitido**.

# ¿Qué integridad se rompe?

**carrera**

| id | nombre |
|---|---|
| 1 | Sistemas |
| 2 | Industrial |

**alumno**

| id | nombre | edad | carrera_id |
|---|---|---|---|
| 1 | Ana | 19 | 1 |
| 1 | Luis | 20 | 2 |
| 3 | Sofía | -4 | 1 |
| 4 | Pedro | 21 | 9 |

| Dato marcado | Integridad rota | Restricción que lo evita |
|---|---|---|
| El `id` 1 de Luis | | |
| La edad `-4` de Sofía | | |
| La `carrera_id` 9 de Pedro | | |

# Tres formas de escribir una restricción

**1. En la misma columna**

al lado del tipo, como en la sesión pasada

```
CREATE TABLE cliente (
   codigo CHAR(10) NOT NULL PRIMARY KEY,
   …
);
```

**2. En una línea aparte**

al final, dentro del `CREATE TABLE`

```
CREATE TABLE cliente (
   codigo CHAR(10) NOT NULL,
   …,
   PRIMARY KEY (codigo)
);
```

**3. En una tabla que ya existe**

se le «agrega» después, con `ALTER TABLE`

```
ALTER TABLE cliente
ADD PRIMARY KEY (codigo);
```

Las tres dan **el mismo resultado**. La 2 es obligatoria cuando la regla abarca **varias columnas**, por ejemplo `PRIMARY KEY (alumno_id, curso_id)`.

# Integridad referencial

# Integridad referencial: no apuntar al vacío

cliente

| id PK | nombre |
|---|---|
| 1 | Ana |
| 2 | Luis |




pedido

| id PK | fecha | cliente_id FK |
|---|---|---|
| 10 | 2026-09-01 | 1 |
| 11 | 2026-09-02 | 2 |
| 12 | 2026-09-02 | 1 |
| 13 | 2026-09-03 | 97 |

**La regla**

Todo valor de una **clave foránea** debe existir como **clave primaria** en la otra tabla. El pedido 13 dice que es del cliente 97… pero ese cliente **no existe**.

Con la `FOREIGN KEY` declarada, el gestor **rechaza** el pedido 13 antes de guardarlo. Sin ella, lo acepta en silencio.

# Formas de crear la relación dentro de `CREATE TABLE`

**1. En la misma columna**

solo `REFERENCES`, al lado del tipo

```
CREATE TABLE pedido (
   id INT PRIMARY KEY,
   cliente_id INT
      REFERENCES
      cliente(id)
);
```

**2. En una línea aparte**

la que usamos la sesión pasada

```
CREATE TABLE pedido (
   id INT PRIMARY KEY,
   cliente_id INT,
   FOREIGN KEY (cliente_id)
      REFERENCES cliente(id)
);
```

**3. Aparte y con nombre**

`CONSTRAINT` le pone nombre

```
CREATE TABLE pedido (
   id INT PRIMARY KEY,
   cliente_id INT,
   CONSTRAINT fk_pedido_cliente
   FOREIGN KEY (cliente_id)
      REFERENCES cliente(id)
);
```

Las tres dicen lo mismo: «`cliente_id` **apunta al** `id` **de** `cliente`». La 3 además le da un nombre a la regla, útil para leer errores o para quitarla después.

## Ojo con esto

En MySQL la forma 1 se acepta pero **no se aplica** en versiones anteriores a la 9.0: no aparece ningún error y la relación simplemente no existe. Usa la 2 o la 3 y funcionará igual en los tres gestores.

# Y si la tabla ya existe: `ALTER TABLE`

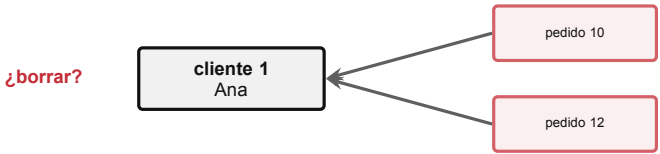
```
ALTER TABLE pedido
ADD CONSTRAINT fk_pedido_cliente
   FOREIGN KEY (cliente_id) REFERENCES cliente(id);
```

| Gestor | ¿Funciona? |
|---|---|
| MySQL | Sí, tal cual. |
| SQL Server | Sí, tal cual. |
| SQLite | **No.** SQLite no deja agregar claves foráneas después: hay que declararlas al crear la tabla. |

**En palabras simples**

Ponerle nombre (`fk_pedido_cliente`) es como etiquetar un cable: cuando el gestor se queje, el mensaje de error dirá **cuál** regla se rompió, y si algún día hay que quitarla, se la llama por su nombre.

# ¿Y si borro un cliente que tiene pedidos?




| Se agrega a la `FOREIGN KEY` | Qué pasa con los pedidos 10 y 12 |
|---|---|
| *(nada)* | El gestor **no deja borrar** a Ana. Es lo más seguro. |
| `ON DELETE CASCADE` | Se borran **también** sus pedidos, en cadena. |
| `ON DELETE SET NULL` | Los pedidos quedan, pero con `cliente_id` vacío. |

```
FOREIGN KEY (cliente_id) REFERENCES cliente(id) ON DELETE CASCADE
```

Se escribe igual en MySQL, SQL Server y SQLite.

# Práctica 2 ¿Se guarda o no?

cliente

| id | nombre |
|---|---|
| 1 | Ana |
| 2 | Luis |

Cada pedido debe ser de un cliente que **exista**. ¿El gestor guarda estos pedidos?

| Pedido nuevo | ¿Se guarda? |
|---|---|
| pedido 20, del cliente 2 | _____ |
| pedido 21, del cliente 5 | _____ |
| pedido 22, del cliente 1 | _____ |

**Pista:** busca el número del cliente en la tabla de la izquierda. Si no está, el pedido **se rechaza**.

# Otras restricciones

# Tres restricciones: NN, UQ y AI

| Sigla | Restricción | En una frase |
|---|---|---|
| NN | `NOT NULL` | La columna **no puede quedar vacía**. |
| UQ | `UNIQUE` | El valor **no se repite** en dos filas. |
| AI | `AUTO_INCREMENT` | El gestor **numera solo**: 1, 2, 3… |

**Cómo se ve en la realidad**

Estas siglas son las casillas que aparecen en MySQL Workbench al diseñar una tabla: PK NN UQ AI. Marcar la casilla es lo mismo que escribir la palabra en el `CREATE TABLE`.

Para cada una veremos: la idea, cómo se escribe en **MySQL**, en **SQL Server** y en **SQLite**, y una práctica.

# NN NOT NULL: no se puede dejar vacío

Un alumno sin nombre no sirve para nada. `NULL` significa «**no hay dato**», y `NOT NULL` lo prohíbe.

**MySQL**

```sql
CREATE TABLE alumno (
    id INT PRIMARY KEY,
    nombre VARCHAR(80)
        NOT NULL
);
```

**SQL Server**

```sql
CREATE TABLE alumno (
    id INT PRIMARY KEY,
    nombre NVARCHAR(80)
        NOT NULL
);
```

**SQLite**

```sql
CREATE TABLE alumno (
    id INTEGER PRIMARY KEY,
    nombre TEXT
        NOT NULL
);
```

```sql
INSERT INTO alumno (id) VALUES (1);   -- sin nombre
```

| | |
|---|---|
| **MySQL** | `Field 'nombre' doesn't have a default value` |
| **SQL Server** | `Cannot insert the value NULL into column 'nombre'` |
| **SQLite** | `NOT NULL constraint failed: alumno.nombre` |

**Se escribe igual en los tres.** Lo que cambia es cómo cada uno redacta el error.

# Práctica 3 Del formulario a la tabla: ¿cuáles son `NOT NULL`?




* obligatorio

Cuando el usuario pulsa **Registrarme**, se guarda una fila en la tabla `usuario`. Decide qué columnas llevan `NOT NULL`:

| Columna | ¿`NOT NULL`? |
|---|---|
| `nombre` | _____ |
| `correo` | _____ |
| `contrasena` | _____ |
| `celular` | _____ |
| `como_nos_conociste` | _____ |

**Pista:** el * del formulario y el `NOT NULL` de la tabla dicen **lo mismo**: sin ese dato, no se guarda.

# UQ UNIQUE: no se puede repetir

Dos alumnos pueden llamarse igual, pero **no pueden tener el mismo correo**.

**MySQL**

```
CREATE TABLE alumno (
   id INT PRIMARY KEY,
   correo VARCHAR(120)
      UNIQUE
);
```

**SQL Server**

```
CREATE TABLE alumno (
   id INT PRIMARY KEY,
   correo NVARCHAR(120)
      UNIQUE
);
```

**SQLite**

```
CREATE TABLE alumno (
   id INTEGER PRIMARY KEY,
   correo TEXT
      UNIQUE
);
```

```
INSERT INTO alumno VALUES (1, 'ana@utp.edu.pe');   -- bien
INSERT INTO alumno VALUES (2, 'ana@utp.edu.pe');   -- rechazada: correo repetido
```

`UNIQUE` **no es** `PRIMARY KEY`**:** una tabla tiene **una sola** clave primaria, pero puede tener **varias** columnas `UNIQUE` (correo, DNI, teléfono…).

**Diferencia curiosa:** una columna `UNIQUE` sin `NOT NULL` admite vacíos. MySQL y SQLite aceptan **muchos** `NULL`; SQL Server, **solo uno**.

# ¿UNIQUE o no?

¿En cuáles columnas tiene sentido prohibir que el valor se repita?

| Tabla | Columna | ¿UQ? | ¿Por qué? |
|---|---|---|---|
| alumno | dni | _____ | _____ |
| alumno | apellido | _____ | _____ |
| producto | codigo_barras | _____ | _____ |
| producto | precio | _____ | _____ |
| auto | placa | _____ | _____ |

**Pista:** imagina que llegan **dos filas** con el mismo valor. ¿Es un error o es normal?

# AI Autoincremento: el gestor numera solo

Nadie quiere recordar cuál fue el último `id`. Con autoincremento, **no se escribe**: el gestor pone el siguiente.

**MySQL**

```
id INT PRIMARY KEY
   AUTO_INCREMENT
```

**SQL Server**

```
id INT IDENTITY(1,1)
   PRIMARY KEY
```

**SQLite**

```
id INTEGER PRIMARY KEY
   AUTOINCREMENT
```

```
INSERT INTO alumno (nombre) VALUES ('Ana');
INSERT INTO alumno (nombre) VALUES ('Luis');
INSERT INTO alumno (nombre) VALUES ('Sofía');
```

| id | nombre |
|---|---|
| 1 | Ana |
| 2 | Luis |
| 3 | Sofía |

`IDENTITY(1,1)` se lee «**empieza en 1, suma de 1 en 1**». En SQLite la columna tiene que ser `INTEGER`, no `INT`. Y en SQL Server **no puedes** escribir el `id` tú mismo: da error.

# ¿Qué número le toca?

La tabla `alumno` tiene autoincremento y ya guardó a Ana (1), Luis (2) y Sofía (3). Ahora:

```
DELETE FROM alumno WHERE id = 3;              -- se va Sofía
INSERT INTO alumno (nombre) VALUES ('Pedro');
```

¿Qué `id` recibe Pedro? _____

**Decide y explica**

¿Por qué crees que el gestor **no reutiliza** el número de una fila borrada? Piensa en una boleta impresa que decía «alumno 3».

# Dos más de dominio: `DEFAULT` y `CHECK`

**DEFAULT**

Si no se indica el dato, **se usa este**.

```
estado VARCHAR(20) DEFAULT 'activo'
```

**CHECK**

El dato tiene que **cumplir una condición**.

```
nota INT CHECK (nota BETWEEN 0 AND 20)
```

```
CREATE TABLE matricula (
   id      INT PRIMARY KEY AUTO_INCREMENT,
   nota    INT CHECK (nota BETWEEN 0 AND 20),
   estado  VARCHAR(20) DEFAULT 'activo'  CHECK (estado IN ('activo', 'retirado'))
);
```

Una nota de `25` se rechaza. Si no se dice el estado, queda `'activo'`. Y el estado solo puede ser uno de esos dos.

**En los tres gestores** se escriben igual (solo cambia el tipo y el autoincremento). **Ojo:** MySQL recién respeta `CHECK` desde la versión 8.0.16; antes lo ignoraba.

# Resumen: la misma regla, en cada gestor

| | MySQL | SQL Server | SQLite |
|---|---|---|---|
| NN No vacío | `NOT NULL` | `NOT NULL` | `NOT NULL` |
| UQ No repetido | `UNIQUE` | `UNIQUE` | `UNIQUE` |
| AI Numerar solo | `AUTO_INCREMENT` | `IDENTITY(1,1)` | `AUTOINCREMENT` |
| Por defecto | `DEFAULT` | `DEFAULT` | `DEFAULT` |
| Validar | `CHECK` | `CHECK` | `CHECK` |
| Clave foránea | `FOREIGN KEY` | `FOREIGN KEY` | `FOREIGN KEY`* |

**Casi todo es idéntico.** La única que cambia de nombre es el autoincremento.

* En SQLite hay que encenderlas con `PRAGMA foreign_keys = ON;` o no se vigilan.

# Práctica 6 Escribe la tabla completa

Escribe en **MySQL** el `CREATE TABLE` de `libro` con estas reglas, y luego marca qué cambiarías para **SQL Server**:

- `id`: entero, se numera solo, es la clave primaria.
- `titulo`: texto de hasta 150 caracteres, obligatorio.
- `isbn`: texto de 13 caracteres, no se puede repetir.
- `paginas`: entero, tiene que ser mayor que 0.
- `disponible`: texto, por defecto `'si'`.
- `autor_id`: entero, obligatorio, apunta a `autor(id)`.

```
CREATE TABLE libro (
   …
);
```

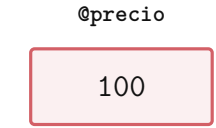
# Preguntas



UTP Universidad Tecnológica del Perú

# Variables

# Una variable es una caja con nombre




nombre arriba, valor adentro

Guarda un valor **por un rato**, para usarlo después sin volver a escribirlo. No es una tabla: **se pierde** al cerrar la conexión.

**MySQL**

```
SET @precio = 100;
SET @igv = @precio * 0.18;

SELECT @precio, @igv,
   @precio + @igv AS total;
```

Basta con `SET`: la variable **nace** al darle valor, sin decir su tipo.

**SQL Server**

```
DECLARE @precio DECIMAL(10,2) = 100;
DECLARE @igv DECIMAL(10,2) = @precio * 0.18;

SELECT @precio, @igv,
   @precio + @igv AS total;
```

Primero se **declara** con `DECLARE` y su tipo. Vive hasta el siguiente `GO`.

**SQLite** no tiene variables en SQL: el valor se escribe directo o lo pasa el programa (por ejemplo, Python).

# Práctica 7 Calcula con variables

Una matrícula cuesta **S/ 850** y hay un descuento de **15 %**. Completa en **MySQL** y luego pásalo a **SQL Server**:

**MySQL**

```
SET @costo = _________;
SET @descuento = @costo * ________;
SELECT @costo - @descuento
   AS a_pagar;
```

**SQL Server**

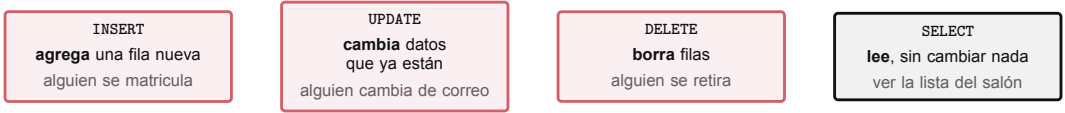
```
___________ @costo DECIMAL(10,2) = 850;
___________ @descuento ____________
   = @costo * 0.15;
SELECT @costo - @descuento AS a_pagar;
```

¿Cuánto debe salir en `a_pagar`? _____

# Primeros pasos en DML

# DML: trabajar con los datos

Hasta ahora usamos **DDL** para construir el envase. El **DML** (lenguaje de manipulación de datos) sirve para **llenarlo, cambiarlo y leerlo**.




## En palabras simples

Es como la agenda del celular: **agregas** un contacto, **editas** su número, lo **eliminas** o simplemente lo **buscas**. Cuatro acciones, cuatro palabras.

`SELECT` a veces se separa como DQL (consulta). Aquí lo vemos junto, porque sin él no podríamos ver lo que guardamos.

# La tabla de ejemplo

```sql
CREATE TABLE alumno (
   id      INT PRIMARY KEY AUTO_INCREMENT,
   nombre  VARCHAR(80) NOT NULL,
   correo  VARCHAR(120) UNIQUE,
   ciclo   INT DEFAULT 1
);
```

alumno

| id | nombre | correo | ciclo |
|---|---|---|---|
| 1 | Ana | ana@utp.pe | 3 |
| 2 | Luis | luis@utp.pe | 1 |
| 3 | Sofía | sofia@utp.pe | 5 |

En los ejemplos que siguen partimos siempre de estas tres filas y vemos **cómo queda la tabla** después de cada instrucción.

**Ojo con esto**

Los ejemplos están en MySQL, pero `INSERT`, `UPDATE`, `DELETE` y `SELECT` **se escriben igual** en SQL Server y en SQLite.

# INSERT: agregar una fila

```
INSERT INTO alumno (nombre, correo, ciclo)
VALUES ('Pedro', 'pedro@utp.pe', 2);
```

- `INSERT INTO` + **tabla** + **columnas** que vas a llenar.
- `VALUES` trae los datos, **en el mismo orden**.
- El texto va entre **comillas simples**; los números, sin comillas.
- El `id` no se escribe: lo pone el autoincremento.

| id | nombre | correo | ciclo |
|---|---|---|---|
| 1 | Ana | ana@utp.pe | 3 |
| 2 | Luis | luis@utp.pe | 1 |
| 3 | Sofía | sofia@utp.pe | 5 |
| 4 | Pedro | pedro@utp.pe | 2 |

```
-- varias filas de una vez
INSERT INTO alumno (nombre) VALUES ('Rosa'), ('Juan');
```

¿Y el `ciclo` de Rosa y Juan? Quedó en `1`, por el `DEFAULT`. ¿Y su correo? `NULL`.

# UPDATE: cambiar lo que ya está

```
UPDATE alumno
SET   ciclo = 2
WHERE id = 2;
```

- UPDATE + **tabla**.
- SET: **qué columna** y **su nuevo valor**.
- WHERE: **a qué filas**. Aquí, solo a Luis.

| id | nombre | correo | ciclo |
|---|---|---|---|
| 1 | Ana | ana@utp.pe | 3 |
| 2 | Luis | luis@utp.pe | 2 |
| 3 | Sofía | sofia@utp.pe | 5 |

**Ojo con esto**

**Sin WHERE, cambian todas las filas.** `UPDATE alumno SET ciclo = 2;` pasa a **todo el salón** al ciclo 2. Por eso MySQL Workbench, en su «modo seguro», rechaza un UPDATE que no filtre por la clave primaria.

# DELETE: borrar filas

```sql
DELETE FROM alumno
WHERE id = 3;
```

- `DELETE FROM` + **tabla**.
- `WHERE`: **cuáles** filas se van.
- Se borra la **fila entera**, no una columna.

| id | nombre | correo | ciclo |
|---|---|---|---|
| 1 | Ana | ana@utp.pe | 3 |
| 2 | Luis | luis@utp.pe | 1 |
| ~~3~~ | ~~Sofía~~ | ~~sofia@utp.pe~~ | ~~5~~ |

**Ojo con esto**

**Sin `WHERE`, se borran todas las filas.** `DELETE FROM alumno;` deja la tabla vacía y **no hay papelera**. Antes de borrar, prueba con un `SELECT` con el mismo `WHERE`.

Y recuerda la integridad referencial: si otra tabla apunta a esta fila, el gestor **no deja borrarla**.

# SELECT: leer sin cambiar nada

```sql
-- todo
SELECT * FROM alumno;
```

| id | nombre | correo | ciclo |
|---|---|---|---|
| 1 | Ana | ana@utp.pe | 3 |
| 2 | Luis | luis@utp.pe | 1 |
| 3 | Sofía | sofia@utp.pe | 5 |

```sql
-- solo algunas columnas y filas
SELECT nombre, ciclo FROM alumno
WHERE ciclo >= 3;
```

| nombre | ciclo |
|---|---|
| Ana | 3 |
| Sofía | 5 |

- `*` significa «**todas las columnas**». Si escribes nombres, salen solo esas.
- `WHERE` filtra **filas**, igual que en `UPDATE` y `DELETE`.
- `SELECT` **nunca** cambia la tabla: puedes ejecutarlo mil veces sin miedo.

# ¿Cómo queda la tabla?

Parte de la tabla `alumno` con Ana (1), Luis (2) y Sofía (3). Ejecuta en orden y dibuja **cómo queda** al final:

```sql
-- 1
INSERT INTO alumno (nombre, ciclo)
VALUES ('Marco', 4);

-- 2
UPDATE alumno SET ciclo = 6 WHERE id = 3;

-- 3
DELETE FROM alumno WHERE id = 1;

-- 4
SELECT nombre FROM alumno WHERE ciclo > 3;
```

| id | nombre | ciclo |
|---|---|---|
| ___ | _____ | _____ |
| ___ | _____ | _____ |
| ___ | _____ | _____ |
| ___ | _____ | _____ |

¿Qué nombres muestra el `SELECT` del paso 4? _____

# Cierre: completa en voz alta

- Los tres tipos de integridad son: _____
- Una clave foránea no puede apuntar a _____
- `NN`, `UQ` y `AI` significan: _____
- En SQL Server, `AUTO_INCREMENT` se escribe _____
- Las cuatro instrucciones del DML son: _____
- Si olvido el `WHERE` en un `UPDATE` o `DELETE`…_____

# Referencias

Coronel, C., Morris, S., & Rob, P. (2011). *Bases de datos: diseño, implementación y administración* (9.ª ed.). Cengage Learning.

Elmasri, R., & Navathe, S. (2007). *Fundamentos de sistemas de bases de datos* (5.ª ed.). Addison Wesley.

Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). *Database system concepts* (7th ed.). McGraw-Hill.




UTP Universidad Tecnológica del Perú